

```
timescale 1ns / 1ps
`default_nettype none
`include "parameters.vh"
```

```
module MEM(
    input wire clk,
    input wire rst_n,
    input wire [31:0] i_Result,
    input wire [31:0] i_DataStore,
    input wire [31:0] i_Pc,
    input wire [2:0] i_Func3,
    input wire i_Load,
    input wire i_MemSrc,
    input wire [1:0] i_ResultSrc,
    input wire [4:0] i_Rd,
    input wire i_RegSrc,
    input wire i_is_amo,
    input wire [4:0] i_amo_funct5, // new
    input wire i_rd_ack,
    input wire [31:0] i_read_data,

    output wire o_RegSrcW,
    output wire [4:0] o_RdW,
    output wire [1:0] o_ResultSrcW,
    output wire [31:0] o_ResultW,
    output wire [31:0] o_WbDataW,
    output wire [31:0] o_Pc4W,

    output wire o_stb,
    output wire o_MemSrc,
    output wire [31:0] o_addr,
    output wire [31:0] o_wr_data
);

wire is_stall;
reg [4:0] Rd_reg;
reg [1:0] ResultSrc_reg;
reg RegSrc_reg;
reg [31:0] Result_reg, WbData_reg, Pc4_reg;

wire [31:0] o_wb_data;
wire [31:0] o_Pc4 = i_Pc + 32'd4;

assign is_stall = !i_rd_ack & ~rst_n;

// === Memory Read Data Extension (Load) ===
```

```

assign o_wb_data = (i_Func3 == `LB) ? {{24{i_read_data[7]}}}, i_read_data[7:0]} :
                  (i_Func3 == `LH) ? {{16{i_read_data[15]}}}, i_read_data[15:0]} :
                  (i_Func3 == `LBU) ? {24'd0, i_read_data[7:0]} :
                  (i_Func3 == `LHU) ? {16'd0, i_read_data[15:0]} :
                  i_read_data;

// === Address and Data for Store/AMO operations ===
assign o_addr = i_Result;
assign o_MemSrc = i_MemSrc;

// === Store Data (handling byte/half/word) ===
assign o_wr_data = (i_Func3[1:0] == 2'b00) ? {24'd0, i_DataStore[7:0]} :
                  (i_Func3[1:0] == 2'b01) ? {16'd0, i_DataStore[15:0]} :
                  i_DataStore;

// === Memory STB ===
assign o_stb = (i_Load || i_MemSrc || i_is_amo) ? 1'b1 : 1'b0;

// === Sequential State ===
always @(posedge clk) begin
    if (~rst_n) begin
        Rd_reg <= 5'b0;
        RegSrc_reg <= 1'b0;
        Result_reg <= 32'b0;
        ResultSrc_reg <= 2'b0;
        WbData_reg <= 32'b0;
        Pc4_reg <= 32'b0;
    end else begin
        Rd_reg <= i_Rd;
        RegSrc_reg <= i_RegSrc;
        ResultSrc_reg <= i_ResultSrc;
        Result_reg <= i_Result;
        WbData_reg <= o_wb_data;
        Pc4_reg <= o_Pc4;
    end
end

end

// === Outputs ===
assign o_RdW = Rd_reg;
assign o_RegSrcW = RegSrc_reg;
assign o_ResultSrcW = ResultSrc_reg;
assign o_ResultW = Result_reg;
assign o_WbDataW = WbData_reg;
assign o_Pc4W = Pc4_reg;

// === Simulation Logging ===

```

```
`ifndef SIM
integer fd;
always @(posedge clk) begin
    #2
    if (o_stb) begin
        fd = $fopen("MEM_log.csv", "ab+");
        $fwrite(fd, "mem:%h\n", o_addr);
        $fclose(fd);
    end else begin
        fd = $fopen("MEM_log.csv", "ab+");
        $fwrite(fd, "\t\n");
        $fclose(fd);
    end
end
`endif

endmodule
```